

Proyecto DIAS: Diseño de un Sistema Conversacional Basado en IA



Sprint 1: Modelo de Dominio

Francisco Iván San Segundo

Alfredo del Val Ramos

Daniel Garcia Salinas

Abril del 2025

Curso 24-25 Ingeniería Informática Valladolid

Índice

1. Introducción al Sprint 1: Modelo de Dominio	2
2. Análisis de Requisitos Funcionales para el Modelo de Dominio	2
2.1. RF-1: Generar texto a partir de una entrada textual	2
2.2. RF-2: Generar imágenes a partir de descripciones	3
2.3. RF-3: Ofrecer respuestas técnicas (ej. código Python)	3
2.4. RF-4: Recibir consultas desde apps de terceros mediante API REST	4
2.5. RF-5: Aceptar entradas desde dispositivos IoT	4
2.6. RF-6: Enviar notificaciones/respuestas a smartwatches y dispositivos móviles	5
2.7. RF-7: Operar desde navegador y adaptarse a diferentes resoluciones	5
2.8. RF-8: Almacenamiento y recuperación del historial de conversaciones . . .	5
2.9. RF-9: Registro de actividad del usuario (logs, estadísticas)	6
2.10. RF-10: Soporte para diferentes idiomas	7
3. Diagrama de Clases del Modelo de Dominio	8

1. Introducción al Sprint 1: Modelo de Dominio

Este primer sprint se enfoca en el desarrollo del **Modelo de Dominio** de la aplicación conversacional basada en Inteligencia Artificial. El objetivo principal es identificar y definir las entidades clave (clasificadores), sus atributos y las relaciones entre ellas, basándose en los requisitos funcionales de la aplicación.

Tal como se indica en el enunciado del proyecto, a partir de la lista de Requisitos Funcionales (RFs), se deben analizar las interacciones y los datos involucrados para construir una representación conceptual del sistema. Esto implica:

- Identificar los principales **conceptos o entidades** que maneja el sistema (por ejemplo: Usuario, Consulta, Respuesta, Imagen Generada, Dispositivo, etc.).
- Definir los **atributos** relevantes para cada una de estas entidades (por ejemplo, para un Usuario: id, nombre, email; para una Consulta: id, contenido, tipo, timestamp).
- Establecer las **relaciones** entre estas entidades (por ejemplo, un Usuario puede realizar muchas Consultas, una Consulta tiene una Respuesta).

Un ejemplo proporcionado es el de "Generar Imágenes a Partir de Descripciones", donde se identifican clasificadores como 'Consulta', 'Descripción' e 'ImagenGenerada'. Este enfoque se aplicará a los 10 requisitos funcionales detallados a continuación para construir un modelo de dominio robusto. El resultado final de este sprint, además de la descripción textual de estos elementos, se materializará en un diagrama de clases que visualice dicho modelo.

2. Análisis de Requisitos Funcionales para el Modelo de Dominio

A continuación, se detallan los requisitos funcionales y el análisis preliminar para la identificación de entidades y atributos del modelo de dominio.

2.1 RF-1: Generar texto a partir de una entrada textual

Descripción del Requisito: Cuando un usuario desea generar texto, se debe interactuar con la aplicación proporcionando una entrada textual inicial. Esta entrada puede ser una pregunta (por ejemplo: ¿Como puedo cocinar x plato?), una frase que la aplicación debe completar (por ejemplo: dime como continua este refran: en abril...), un request de información sobre un tema (por ejemplo: proporcioname más información respecto del águila real)...

Esta entrada textual equivaldría a una nueva consulta en el sistema, cuya respuesta debe de ser de tipo "texto". El sistema generará de forma automática un contenido que equivaldría a la respuesta. Esta será enviada al usuario con el texto generado y un estado

del proceso (éxito o fallo). En caso de fallo, se mostraría simplemente este estado con un texto predefinido de error. Aparte de la respuesta, se generarán ciertos metadatos como el id de la respuesta, la consulta a la que esta asociada o el momento en el que se generó la respuesta.

2.2 RF-2: Generar imágenes a partir de descripciones

Descripción del Requisito: Cuando el usuario interactúa con la aplicación y desea generar una imagen, el primer paso consiste en enviar una descripción textual que explica de manera natural lo que desea obtener (por ejemplo, una ciudad futurista al atardecer.º un robot pintando un cuadro en un museo”). Esta descripción se recibe y se registra como parte de una nueva consulta en el sistema. Una vez almacenada la consulta, el sistema de backend analiza la descripción utilizando un motor de IA, que puede ser un modelo de generación de imágenes (por ejemplo, DALL·E, Stable Diffusion u otro). El modelo interpreta la descripción para extraer los conceptos relevantes y generar una imagen sintética que represente visualmente lo solicitado. El proceso de generación también produce metadatos adicionales que se deben registrar en el sistema. Entre ellos se incluye un identificador único de la imagen generada, la fecha y hora de creación, y una posible clasificación temática de la imagen basada en palabras clave detectadas en la descripción (por ejemplo: naturaleza, tecnología, fantasía...). Cada imagen generada se asocia directamente a la consulta original, de manera que el historial de interacciones del usuario pueda ser reconstruido posteriormente. Además, la respuesta enviada al usuario debe contener una referencia al archivo o recurso de la imagen, junto con el estado final del proceso (éxito o fallo en la generación).

2.3 RF-3: Ofrecer respuestas técnicas (ej. código Python)

Descripción del Requisito: Un usuario, que suele ser desarrollador, necesita interactuar con la aplicación para obtener una solución técnica a un problema que le ha surgido, o simplemente resolver una duda técnica de carácter específico. Esto puede ser por ejemplo, un código en un lenguaje de programación que cumpla una función específica (una función en python que ordene un vector) o consideraciones técnicas sobre conexiones/inicializaciones de cierto software (conexión con una base de datos).

El usuario debe introducir sus requisitos técnicos o simplemente una descripción del problema, siendo esto una consulta nueva. El sistema debe de identificar que esta es una consulta de tipo técnico y debe prepararse para crear una respuesta de tal carácter. Además de la explicación técnica, si la aplicación lo considera conveniente y la consulta del usuario no indica lo contrario, se pueden generar comentarios explicativos en el código, explicaciones a mayores del funcionamiento general o incluso ejemplo.

2.4 RF-4: Recibir consultas desde apps de terceros mediante API REST

Descripción del Requisito: El sistema debe permitir que aplicaciones externas, desarrolladas por usuarios externos, interactúen con sus capacidades de generación de IA. Para ello, se expondrá una API REST segura y bien documentada. Una aplicación cliente de terceros que desee utilizar el servicio deberá primero autenticarse, utilizando un API Key o un token OAuth proporcionado por nuestro sistema tras un proceso de registro de la aplicación. Una vez autenticada, la aplicación cliente podrá enviar solicitudes HTTP a los endpoints designados de la API (por ejemplo, `/api/v1/generate/text`, `/api/v1/generate/image`). Estas solicitudes contendrán los mismos datos que una consulta normal (descripción, tipo de contenido deseado, etc.), probablemente en formato JSON en el cuerpo de la petición. El sistema procesará estas solicitudes de forma idéntica a las consultas internas y devolverá la respuesta (texto generado, URL de imagen, código) también en formato JSON, junto con los códigos de estado HTTP apropiados. Se registrará el uso de la API por cada aplicación cliente, incluyendo el número de solicitudes, los endpoints accedidos y posibles errores, para fines de monitoreo, facturación (si aplica) o limitación de tasa (rate limiting).

2.5 RF-5: Aceptar entradas desde dispositivos IoT

Descripción del Requisito: El sistema debe permitir entradas desde dispositivos IoT (comandos por voz, sensores, ...). Para ello, el primer paso consiste en que el dispositivo envíe estos datos al sistema, esta entrada se recibe y se registra, identificando al dispositivo de origen y el tipo de dato (voz, valor numérico, estado booleano, etc.).

Una vez que se ha almacenado la entrada, el sistema de backend la procesa. En caso de que se trate de un comando de voz, un motor de Reconocimiento Automático de Habla (ASR) lo convierte a texto y un sistema de Comprensión del Lenguaje Natural interpreta la intención del usuario y extrae las entidades relevantes (por ejemplo, acción: `."encoder"`, objeto: `"luz"`, ubicación: `"salón"`). Si se trata de datos de sensores, el sistema lo analiza para determinar si desencadena alguna regla o acción predefinida.

En el proceso de interpretación se genera una serie de metadatos que se deben registrar como: un identificador único del evento IoT, el ID del dispositivo, la fecha y hora de recepción, la transcripción del comando de voz (si aplica) y el valor o estado del sensor.

Cada evento o comando se asocia a un dispositivo y a un usuario permitiendo reconstruir el historial de interacciones y ejecutar las acciones en el sistema.

Se puede enviar una confirmación al dispositivo IoT indicando el procesamiento del comando (recibido, ejecutado con éxito, o error)

2.6 RF-6: Enviar notificaciones/respuestas a smartwatches y dispositivos móviles

Descripción del Requisito: El sistema debe permitir enviar notificaciones/respuestas a smartwatches y dispositivos móviles. Para ello, el primer paso consiste en determinar el contenido del mensaje y el dispositivo de destino, como un smartwatch o un teléfono móvil. Esta necesidad se origina internamente como un resultado de un proceso completado, una regla activada por un evento IoT, o una respuesta a una consulta previa del usuario.

Una vez que se ha definido el mensaje y el destinatario, el sistema de backend formatea la notificación adaptándola a las características del dispositivo final y se selecciona el canal de entrega apropiado, como los servicios de notificación push (APNS para Apple, FCM para Google) o mensajería interna de la aplicación.

El proceso de envío, genera metadatos que se registran en el sistema. Estos incluyen un identificador único de la notificación, la fecha y hora de envío, el tipo de notificación (informativa, alerta, respuesta) y el estado de entrega (enviada, recibida, fallida).

La respuesta enviada al dispositivo del usuario contiene el mensaje formateado y, en algunos casos, opciones para que el usuario interactúe directamente desde la notificación (por ejemplo, “Ver detalles”, “Descartar”).

2.7 RF-7: Operar desde navegador y adaptarse a diferentes resoluciones

Descripción del Requisito: El sistema debe permitir operar desde navegador y operar a diferentes resoluciones. Para ello, el primer paso consiste en que el navegador envíe una solicitud al servidor de la aplicación para cargar la interfaz. El sistema detecta las características del dispositivo, principalmente el tipo de navegador y la resolución.

Una vez que se ha recibido la solicitud, el sistema de backend sirve los componentes de la interfaz de usuario (HTML, CSS, JavaScript) que están diseñados para ser responsivos, lo que significa que se ajustan automáticamente para ofrecer una experiencia óptima en la resolución detectada utilizando para ello, técnicas como: media queries, layouts flexibles (Flexbox/Grid) y optimización de imágenes.

Se registran metadatos de la sesión, como el tipo de dispositivo, el navegador utilizado y las interacciones del usuario dentro de la interfaz (navegación, clics, envío de formularios).

La respuesta al usuario es la propia interfaz web interactiva y adaptada con un estado final de carga exitosa, o en caso de problemas, un mensaje de error apropiado.

2.8 RF-8: Almacenamiento y recuperación del historial de conversaciones

Descripción del Requisito: El sistema debe permitir almacenar y recuperar el historial completo de conversaciones de cada usuario.

Para ello, el primer paso consiste en asignar un identificador único de sesión cuando el usuario inicia una nueva conversación (chat). Cada mensaje que se intercambia—ya sea del usuario, de la IA o del propio sistema—se registra en la base de datos con su timestamp y el rol del emisor.

Una vez almacenados los mensajes, el backend organiza la información en hilos de conversación (threads) asociados al usuario; cada hilo mantiene metadatos como fecha de inicio, fecha de cierre y número de mensajes.

Estos metadatos permiten reconstruir la conversación en el orden correcto y filtrar por fecha, tema o estado (abierta, cerrada).

Para la recuperación, el cliente (web, móvil o API externa) envía la petición indicando el `conversation_id` o un rango de tiempo. El backend serializa la conversación a un formato eficiente (JSON paginado o streaming) y lo devuelve al frontend, que lo renderiza en la interfaz de chat.

El proceso genera metadatos adicionales: `conversation_id`, `last_message_at`, `num_tokens`, tamaño en bytes y estado de la consulta (completa / truncada). Estos datos se almacenan para análisis posteriores y para aplicar políticas de retención o borrado conforme al RGPD.

2.9 RF-9: Registro de actividad del usuario (logs, estadísticas)

Descripción del Requisito: El sistema debe registrar toda la actividad relevante (inicios de sesión, consultas, errores y eventos IoT) con fines de auditoría y analítica.

Cuando se produce cualquier evento, el backend crea un `RegistroActividad` con un identificador único, la cuenta de usuario y el tipo de evento (LOGIN, QUERY, ERROR, NOTIFICATION, etc.).

Cada registro almacena: timestamp en UTC, IP de origen, dispositivo, endpoint invocado, payload anonimizado (hash) y el código de respuesta (éxito, validación fallida, rate-limit...).

Para eventos de sistema (p. ej. caída de un microservicio) se añade severidad (INFO, WARN, ERROR) y el componente afectado.

Periódicamente (cron o serverless job) se agregan estos registros en `EstadisticasSistema`: n° de consultas por minuto, latencia media, tokens generados, ratio de errores 4xx/5xx, etc.

Los agregados se almacenan en una base analítica (Time-Series DB o warehouse) y se exponen vía dashboard para el equipo DevOps o, opcionalmente, para el propio usuario en su perfil.

La API de métricas ofrece endpoints `/metrics/daily/{userId}` y `/metrics/system/last24h` que devuelven la información en JSON o Prometheus format.

Todos los logs cumplen con la política de retención y pueden ser anonimizados/borrados bajo petición del usuario.

2.10 RF-10: Soporte para diferentes idiomas

Descripción del Requisito: El sistema debe funcionar en múltiples idiomas, tanto en la interfaz como en la generación de contenido.

Tras detectar el idioma preferido (cabecera Accept-Language, ajuste en la cuenta o selección manual), el backend carga los recursos de localización (archivos *.po / JSON i18n) correspondientes.

Las cadenas de la UI se sirven ya traducidas; los mensajes generados por IA se solicitan al modelo con el prompt enriquecido con la instrucción *You must answer in idioma¿*.

Si el idioma es de escritura RTL (árabe, hebreo), la capa de presentación aplica las clases CSS apropiadas y cambia la dirección del texto.

Se registran metadatos en la entidad PreferenciaIdioma: user_id, codigo_iso, fecha de selección y confidence score (si se detectó automáticamente).

Cuando el usuario cambia de idioma, se dispara un evento LANG_CHANGE que se guarda en el RegistroActividad (ver RF-9) para estadísticas de localización.

Para contenido estático (FAQ, política de privacidad, prompts de onboarding) se mantiene un CDN con rutas /es/, /en/, /fr/, etc. y fallback al idioma por defecto si no existe traducción.

El estado final de la solicitud incluye el lang efectivo y, si la traducción automática falla, un mensaje de error localizable.

3. Diagrama de Clases del Modelo de Dominio

